PART 2 · CODING AGENT WORKSHOP

Give a coding agent useful memory

With Redis Agent Memory and Redis Agent Playbook

Samuel Agbede · Bank of America Agent Memory workshop

RECAP · PART 1

An agent is a model plus a harness

01 · THE AGENT LOOP

The model decides: call a tool, or answer. Your code runs the tool and sends the result back

02 · TOOL DESIGN

Small, read-only tools with schemas. Capped outputs, errors returned as data

03 · PRODUCTION PATTERNS

Sandboxed paths, a step budget, validated output, a person approves changes

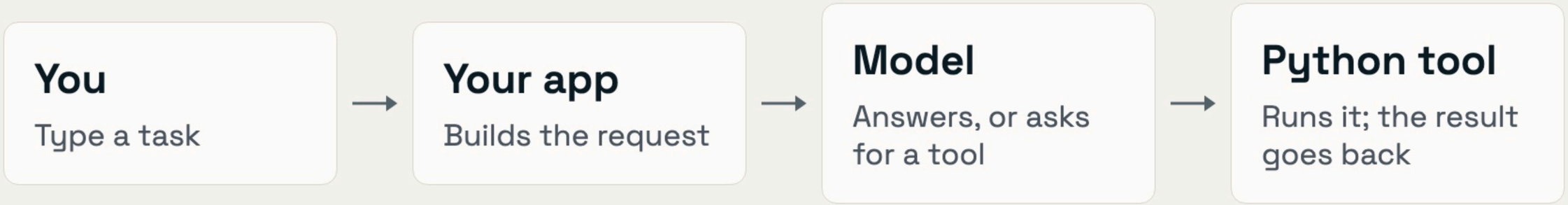
04 · TESTING AGENTS

Fake-model tests on every commit, task evals on every change

Within a run, the context window is its memory. Between runs? That's today.

RECAP · PART 1

The agent loop in one picture



The tool result goes into the next request. The loop repeats until the model answers.

The model never touches your files. Your code does.

RECAP · PART 1

Every turn, your app builds the request

THE MODEL REQUEST

System and project instructions
How to behave, plus AGENTS.md

Conversation history
Everything said in this session

The new message
What you just typed

The model knows only what is in this request.

Everything today is a decision about what else belongs here, and where it comes from.

TODAY'S QUESTION

What should a coding agent know, where does that knowledge come from, and who owns it?

Each section adds one source of knowledge, and the owner gets wider each time.

WHERE KNOWLEDGE LIVES

Four sources, four owners

Source	Owner	Where it lives
The conversation	This session	Process memory; gone after a new session
Project instructions	The project	AGENTS.md in the repository
Personal preferences	You	Redis Agent Memory, learned from your work
Team procedures	The organisation	Redis Agent Playbook, published and versioned

HOW TODAY RUNS

A demo, then five minutes for you

Minutes	Section	Demo	Your turn
0–4	Recap	4	—
4–17	1 · The amnesiac and notes on the desk	8	5
17–32	2 · Capture with Redis Agent Memory	10	5
32–47	3 · Recall in a fresh session	10	5
47–64	4 · The team's way: Redis Agent Playbook	12	5
64–75	5 · Trust, but verify	6	5
75–90	Discussion and buffer	—	—

All the code is already in agent.py. Each Your turn switches on one flag and reads the lines it turns on.

01

The amnesiac and notes on the desk

A preference works in one session and is gone in the next. Writing it down fixes that, at a cost.

8-MINUTE DEMO · 5 MINUTES FOR YOU

01 · THE AMNESIAC

A new session sends a different request

SESSION 1

System and project instructions

Conversation

“By the way, I like Google-style docstrings on my functions.”

Task: a URL slug function

/new-session →

SESSION 2

System and project instructions

Conversation

Empty. The preference is not sent.

Task: the same slug function

The model didn’t forget. The app stopped sending it.

01 · NOTES ON THE DESK

Write it down, and it works every time

PROJECT/AGENTS.MD

```
# Report tools
Python with type hints. Keep functions
small and readable.
Use Google-style docstrings for functions.
```



AGENTS.md is reloaded into every session, so the rule is always there.

01 · WHAT IT COSTS

Writing it down works, but…

Someone had to write it

Most preferences come up in passing and never get turned into a rule

It applies to everyone

A teammate who likes NumPy-style docstrings now gets yours

It only grows

Every rule rides along in every request, even FizzBuzz

Nothing expires

Change your mind and you have to remember to edit the file

Doing it well takes three jobs. That's what Redis Agent Memory is for.

YOUR TURN · 5 MINUTES

Watch a preference disappear

AT THE YOU > PROMPT

1. Mention your docstring preference, then ask for the slug function
2. /new-session, ask for the slug function again, then /view-context
3. Ask the agent to add the preference to AGENTS.md, /new-session, ask for the slug function again
4. Ask the agent to remove the docstring line from AGENTS.md

LOOK FOR

- ✓ After step 2: no Google-style docstring, and no preference in the request
- ✓ After step 3: Google-style again, with the rule in the first message

Prompts to copy: workshop guide, section 2

02

Capture with Redis Agent Memory

Record the conversation and let Redis Agent Memory do the jobs you were doing by hand: decide what to keep, and keep it for the right time.

10-MINUTE DEMO · 5 MINUTES FOR YOU

Three jobs you were doing by hand

EXTRACTION

What's worth keeping

Learned from the conversation, guided by the coding_preference type and screened for sensitive data

This section

LIFECYCLE

How long, and changing your mind

Retention set on the store; new facts update old ones; forget in one step

This section and the next

RETRIEVAL

Only what fits the task

Searched by task, owner and type, at most five, so nothing is dumped in every request

The next section

02 · REDIS AGENT MEMORY

Record events, extract memories, search later

1 · YOUR APP WRITES

Session events

Short-term memory: who said what, in which session

```
add_session_event()
```

→

2 · THE SERVICE LEARNS

Extraction

Runs in the background; the store's custom type says what to keep

```
coding_preference
```

→

3 · YOUR APP SEARCHES

Long-term memories

Found by owner and type, searched with the new task, added to the next request

```
search_long_term_memory()
```

Your app records the work. Redis Agent Memory learns from it.

02 · BEFORE WE START

How the memory is set up

RETENTION

1 day · 7 days

Sessions are kept for a day, long-term memories for a week

EXTRACTION

Every minute

Runs in the background while a session is active

CUSTOM TYPE

`coding_preference`

Keep only what the user says about how they want code written; ignore what the assistant did

Store settings, not code: whoever runs the store decides what it learns.

02 · SENSITIVE DATA

Screened before it’s kept

YOU SAY

In test fixtures, use my email jane.doe@example.com as the sample address.

→

LONG-TERM MEMORY · EMAIL: REDACT

Uses [REDACTED] as the sample address in test fixtures.

YOU SAY

Use my GitHub token ghp_… in the examples.

→

LONG-TERM MEMORY · API-TOKEN: DROP

Nothing. The whole memory is dropped.

Built-in detectors: card numbers, email, IP addresses, phone, US SSNs. Add your own as a regex, or as a plain-language rule.

The session keeps the raw text until it expires. Don’t capture what you couldn’t keep for a day.

02 · CAPTURE

Keep who said what, in which session

```
if CAPTURE:
    services.memory.add_session_event(
        session_id=session.session_id,
        actor_id=session.owner,
        role=models.MessageRole.USER,
        content=[models.Text(text=task)],
        created_at=datetime.now(timezone.utc),
    )
```

role

USER or ASSISTANT, so extraction learns your preferences, not the model's suggestions

actor_id

Your stable owner ID, or coding-assistant

session_id

Groups one conversation's events

In agent.py's handle: your message before the model is called, the answer after it. Tool calls aren't recorded.

02 · PREDICT

Stored is not the same as learned

/events

Recorded

Your messages as USER events and the answers as ASSISTANT events, for this session

/memories

Maybe not yet

Extraction runs in the background. We come back to it after the next demo.

/sessions · /resume

Whole conversation back

Short-term memory replays everything. Useful, but you don't want every old chat in every request.

Sessions keep everything. Long-term memory keeps what matters.

YOUR TURN · 5 MINUTES

Capture your own exchange

IN AGENT.PY, THEN RESTART

```
CAPTURE = True
save, exit, then:
uv run --group iris python main.py
```

THEN AT YOU >

1. Mention your docstring preference and ask for the slug function
2. /events
3. /new-session, /sessions, then /resume 1

LOOK FOR

- ✓ Your messages and the agent's answers in /events
- ✓ After /resume, the earlier conversation is back

Leave /memories for later: extraction needs a few minutes.

03

Recall in a fresh session

Bring back only what fits the task, and forget what you no longer want.

10-MINUTE DEMO · 5 MINUTES FOR YOU

03 · PREDICT

Search by owner, not by session

OWNER: LEARNER-A · STABLE ACROSS SESSIONS

Session 1 · earlier

You mentioned Google-style docstrings; it was extracted as a coding_preference

Session 2 · now

A new task arrives: a de-duplication function

Filter on session 2

Finds nothing. The preference came from session 1.

✓ **Filter on the owner**

Finds the preference, whichever session it came from.

03 · RECALL

The new task is the search query

```
if RECALL:
    found = services.memory.search_long_term_memory(request={
        'text': task,
        'filter_': {
            'owner_id': {'eq': session.owner},
            'memory_type': {'eq': 'coding_preference'},
        },
        'limit': 5,
    })
    preferences = [item.text for item in found.items]
```

`text`

The task itself, so results fit what you are doing now

`owner_id`

The stable owner, never the new session

`memory_type`

Our custom type: only what you said, never what the agent did

Section 4 in agent.py's handle, run fresh for every task.

03 · RECALL

What was found becomes one short message

```
messages = [{'role': 'system', 'content': SYSTEM + agents_md}]
recalled = []
if preferences:
    recalled.append("What you know about this user's preferences:\n" +
bullets)
if skill:
    recalled.append(f"Team procedure ({skill.name}, version
{skill.version}):\n" + rules)
if recalled:
    messages.append({'role': 'system', 'content':
'\n\n'.join(recalled)})
messages += session.history + [{'role': 'user', 'content': task}]
```

THE FRESH REQUEST

- System prompt and AGENTS.md
- **What you know about this user's preferences**
- Conversation: this session only
- New task

Rebuilt for every task, so stale results never pile up in the conversation.

THE POINT OF SECTION 3

Memory isn’t the model remembering. It’s your application choosing what to put back in context.

03 · LIFECYCLE

Change your mind

MONDAY'S SESSION

I like Google-style docstrings.

THE MEMORY

Prefers Google-style docstrings.

FRIDAY'S SESSION

Actually, I've switched to NumPy-style docstrings.

THE SAME MEMORY, UPDATED

Prefers NumPy-style docstrings.

Extraction checks what it already knows: one memory, updated, not two that disagree.

03 · LIFECYCLE

Or forget it

WITH AGENTS.MD

Remember to edit the file

Until someone does, old and new rules sit side by side, for everyone



WITH REDIS AGENT MEMORY

Forget it, for you only

```
/forget b6c0a46e
```

Next task: Recalled: nothing

Retention expires memories by itself; forgetting one is a single step.

YOUR TURN · 5 MINUTES

Recall your preference

IN AGENT.PY, THEN RESTART

```
RECALL = True
save, exit, then:
uv run --group iris python main.py
```

THEN AT YOU >

1. /memories
2. Ask for the de-duplication function, then /view-context
3. Forget the preference in the Agent Memory panel and ask again

LOOK FOR

- ✓ Your preference as a coding_preference, next to built-in semantic and episodic memories
- ✓ “What you know about this user’s preferences” in the second message

After forgetting: Recalled: nothing. Mention the preference again before moving on.

The lines it switches on: agent.py · handle · Section 4 and “Build the request”

04

The team's way: Redis Agent Playbook

Redis Agent Memory learned how you like code written. How the team must validate a report is a different kind of knowledge.

12-MINUTE DEMO · 5 MINUTES FOR YOU

04 · FROM PERSONAL TO ORGANISATIONAL

A preference is learned. A procedure is published.

REDIS AGENT MEMORY

Personal preference

- Learned from your conversations
- Scoped to one owner
- “I like Google-style docstrings”

REDIS AGENT PLAYBOOK

Team procedure

- Authored and reviewed
- Published once, versioned
- Same version for every agent
- “How reports are validated”

Publish once, retrieve everywhere



Every agent loads the same published version, with a source it can cite.

Attendees only read; one publish serves the whole room.

04 · RETRIEVE

Use the procedure only when it fits the task

```
SKILL_MIN_SCORE = 0.7

if PLAYBOOK:
    skill = services.playbook.retrieve(
        task, min_score=SKILL_MIN_SCORE,
    )
```

HOW WELL REPORT-VALIDATION FITS

> “Write a FizzBuzz function.” · 0.58 · not used

> “…a URL slug function.” · 0.55 · not used

> **“What are our rules for validating report rows?” · 0.91 · used**

Search always returns its closest skills. The threshold keeps the team procedure out of tasks it has nothing to do with.

04 · OPTIONAL DEMO

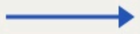
Change the procedure; every agent gets version 2

BEFORE

Version 1

`entry: ent_team`

The rules every attendee retrieved today



AFTER /PUBLISH-SKILL

Version 2

`entry: ent_team`

Adds a rule, for example: “Amounts have at most two decimal places.”

Same entry ID, new version. The next /skills in every workspace shows it.

04 · PROMOTION

From many conversations to one approved answer

AGENT MEMORY

Many people ask

“Can I use real customer data in test fixtures?” Each agent answers its own way

→

EXTRACTION

A proposal

A drafted FAQ, with the sessions it came from as evidence

→

A PERSON

Reviewed, accepted

The answer rewritten to the approved one; a draft FAQ, not yet served

→

PUBLISH

Every agent

Gets the same answer and version, but only when it fits the task

Memory learns for you. Promotion turns what many asked into what the team approved.

Nothing reaches anyone’s agent until a person publishes it.

YOUR TURN · 5 MINUTES

Retrieve the team's procedure

IN AGENT.PY, THEN RESTART

```
PLAYBOOK = True
save, exit, then:
uv run --group iris python main.py
```

THEN AT YOU >

1. /skills
2. Ask for a FizzBuzz function
3. Ask: What are our rules for validating report rows?
4. After the team answer is published: Can I put real customer records in my test fixtures?

LOOK FOR

- ✓ /skills: report-validation, same version as the instructor
- ✓ FizzBuzz: Team procedure: none. Report rules: report-validation

Only a task the procedure fits gets it. /view-context shows it in the second message.

The lines it switches on: agent.py · handle · Section 5

05

Trust, but verify

All three sources guide one task. The project's tests decide whether it worked.

6-MINUTE DEMO · 5 MINUTES FOR YOU

05 · ONE TASK, THREE SOURCES

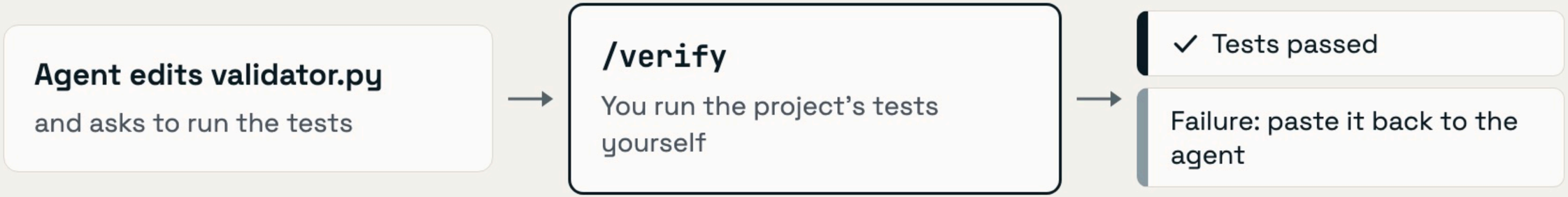
Every piece of the request has an owner

Project instructions: “Python with type hints.”	AGENTS.md · the project
What you know about this user: Google-style docstrings	Redis Agent Memory · you
Team procedure: report-validation, version 1	Redis Agent Playbook · the organisation
Conversation: only this task	This session

Nothing from earlier sessions was replayed.

05 · VERIFY

The tests decide, not the agent



“Tests pass” from the agent is a claim. Your own test run is evidence.

Whether it used your docstring style is still your call.

YOUR TURN · 5 MINUTES

Build and verify the validator

AT THE YOU > PROMPT

1. /new-session
2. Ask for the validator; approve the test run
3. /view-context
4. /verify

LOOK FOR

- ✓ All three sources in the latest request
- ✓ Tests passed.
- ✓ A rule from the team procedure, and your docstring style

Out of time? solutions/validator.py is a reference, not an agent-generated result

WHAT WE BUILT

Three sources, three owners

THE PROJECT

Project convention

“Python with type hints.”

AGENTS.md, reviewed with the code

YOU

Personal preference

“I like Google-style docstrings.”

Redis Agent Memory, learned from your work

THE ORGANISATION

Organisational rule

How reports are validated

Redis Agent Playbook, published and versioned

Pick one of each from your own team.

DISCUSSION · 10 MINUTES

Taking this to production

Who can read a memory?

Owner filters select records. Isolation comes from stores, credentials and namespaces.

Can we correct or delete it?

Redis Agent Memory has update and bulk-delete operations for long-term memories.

What about sensitive data?

Extraction learns from what you capture. Decide what to capture; prompts aren't redaction.

Who approves a procedure?

Playbook separates creating a version from publishing it. Only the publisher needs write access.

How do changes roll out?

Agents load the current published version; its source and version appear in every request.

In your team

Which agent knowledge needs an owner, a version and an audit trail?

Three sources, three owners, each one checkable.

Questions?